

Lecture 6: Sorting Algorithms

Sorting Algorithms

- **Sorting** refers to arranging data in a particular format. Sorting algorithm specifies the way to arrange data in a particular order. Importance of sorting lies in the fact that data searching can be optimized to a very high level if data is stored in a sorted manner.
- Some of the examples of sorting in real life scenarios are following.
 - **Telephone Directory** – Telephone directory keeps telephone no. of people sorted on their names. So that names can be searched.
 - **Dictionary** – Dictionary keeps words in alphabetical order so that searching of any word becomes easy.

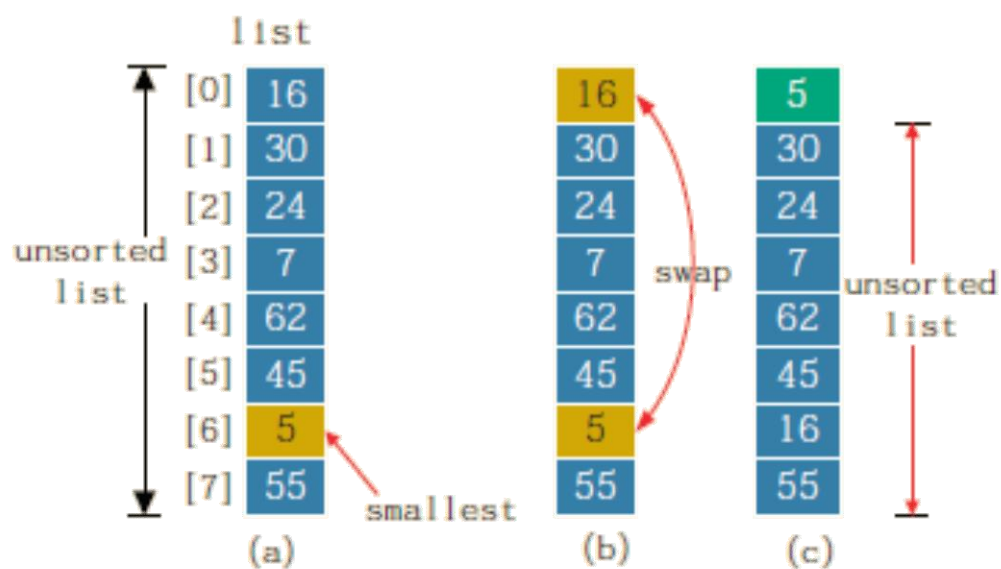
Selection Sort

- The selection sort algorithm sorts a list by selecting the smallest element in the unsorted portion of the list and then moving this smallest element to the top of the unsorted list. The first time, we locate the smallest item in the entire list; the second time, we locate the smallest item in the list starting from the second element in the list, and so on.

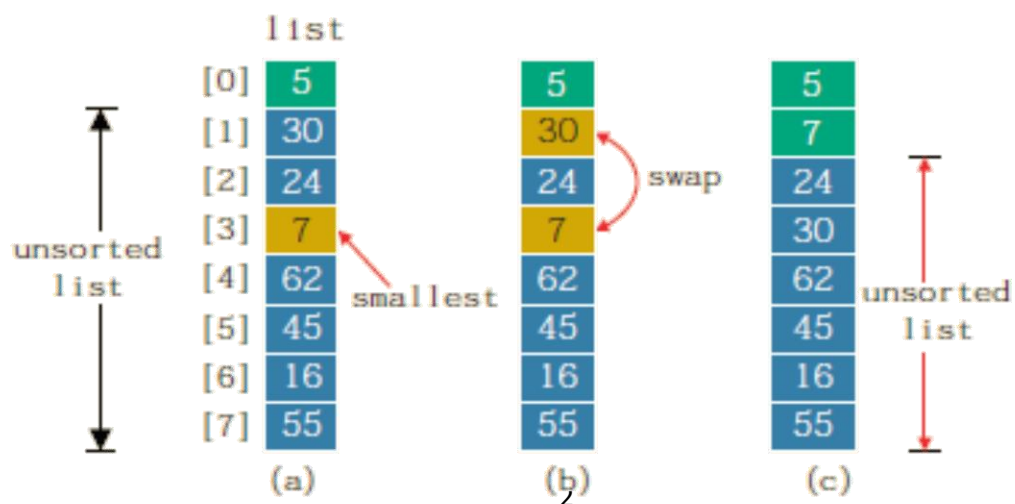
```
void selectionSort(elemType list[], int length)
{
    int loc, minIndex;
    for (loc = 0; loc < length; loc++)
    {
        minIndex = minLocation(list, loc, length - 1);
        swap(list, loc, minIndex);
    }
} //end selectionSort
```

Selection Sort

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
list	16	30	24	7	62	45	5	55



	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
list	16	30	24	7	62	45	5	55



Bubble Sort

- In a series of $n - 1$ iterations, the successive elements $\text{list}[\text{index}]$ and $\text{list}[\text{index} + 1]$ of the list are compared. If $\text{list}[\text{index}]$ is greater than $\text{list}[\text{index} + 1]$, then the elements $\text{list}[\text{index}]$ and $\text{list}[\text{index} + 1]$ are swapped.
- It follows that the smaller elements move toward the top, and the larger elements move toward the bottom.

```
void bubbleSort(elemType list[], int length)
{
    for (int iteration = 1; iteration < length; iteration++)
    {
        for (int index = 0; index < length - iteration; index++)
        {
            if (list[index] > list[index + 1])
            {
                elemType temp = list[index];
                list[index] = list[index + 1];
                list[index + 1] = temp;
            }
        }
    }
} //end bubbleSort
```

Bubble Sort Example 1:

Illustrate the first pass of a bubble sort of the following array:

5, 1, 2, 4, 3, 7, 6

Solution:

First Pass							
							
5	1	2	4	3	7	6	Swap
1	5	2	4	3	7	6	Swap
1	2	5	4	3	7	6	Swap
1	2	4	5	3	7	6	Swap
1	2	4	3	5	7	6	No Swap
1	2	4	3	5	7	6	Swap
1	2	4	3	5	6	7	

Bubble Sort Example 2:

Illustrate the first two passes of a bubble sort of the following array.

The first two passes of a bubble sort of an array of five integers

(a) Pass 1

Initial array:

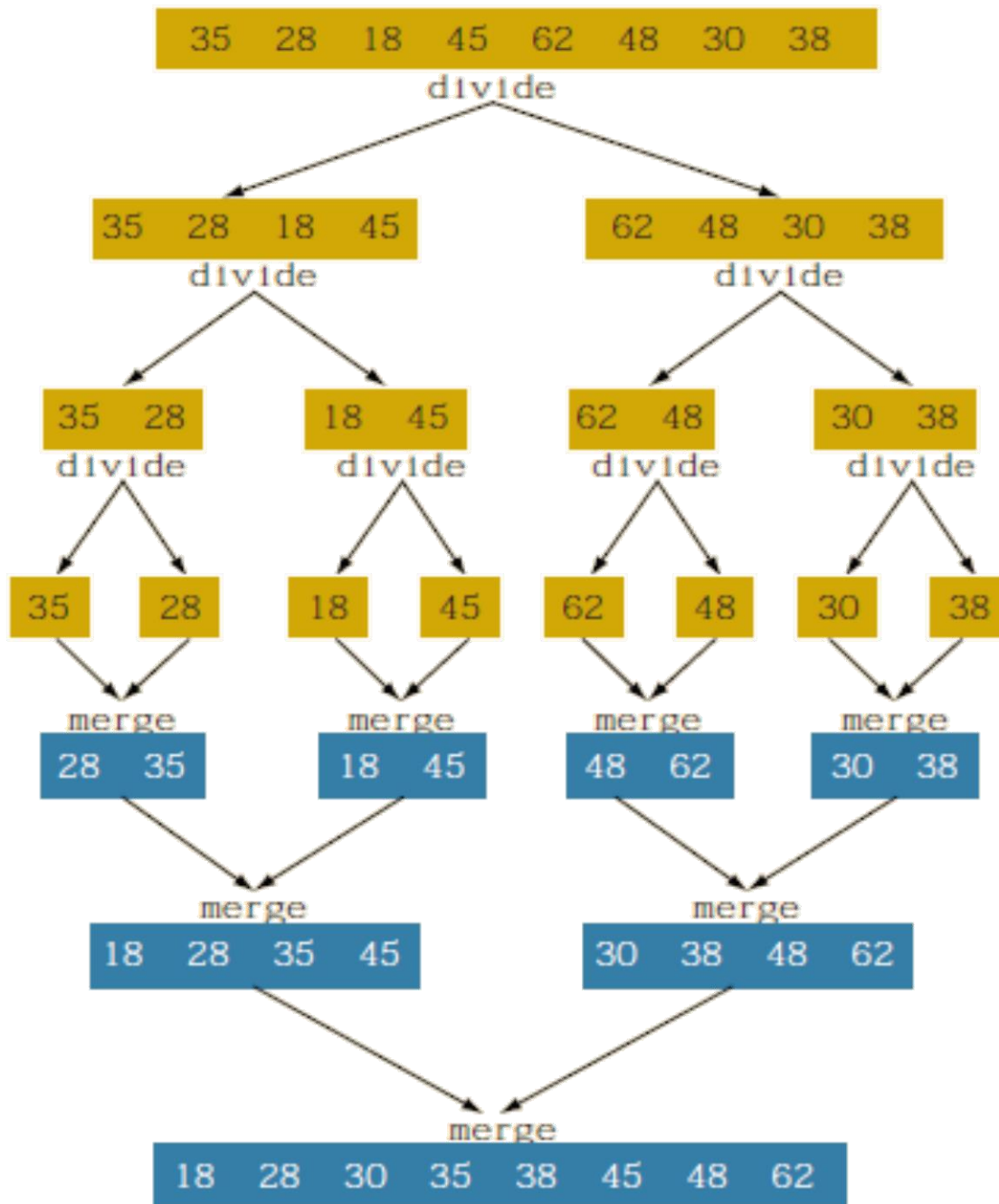
29	10	14	37	13
10	29	14	37	13
10	14	29	37	13
10	14	29	37	13
10	14	29	13	37

(b) Pass 2

10	14	29	13	37
10	14	29	13	37
10	14	29	13	37
10	14	13	29	37

Merge Sort

- The merge sort algorithm uses the divide-and-conquer technique to sort a list. Merge sort first divides the array into equal halves and then combines them in a sorted manner.



Merge Sort Example 2

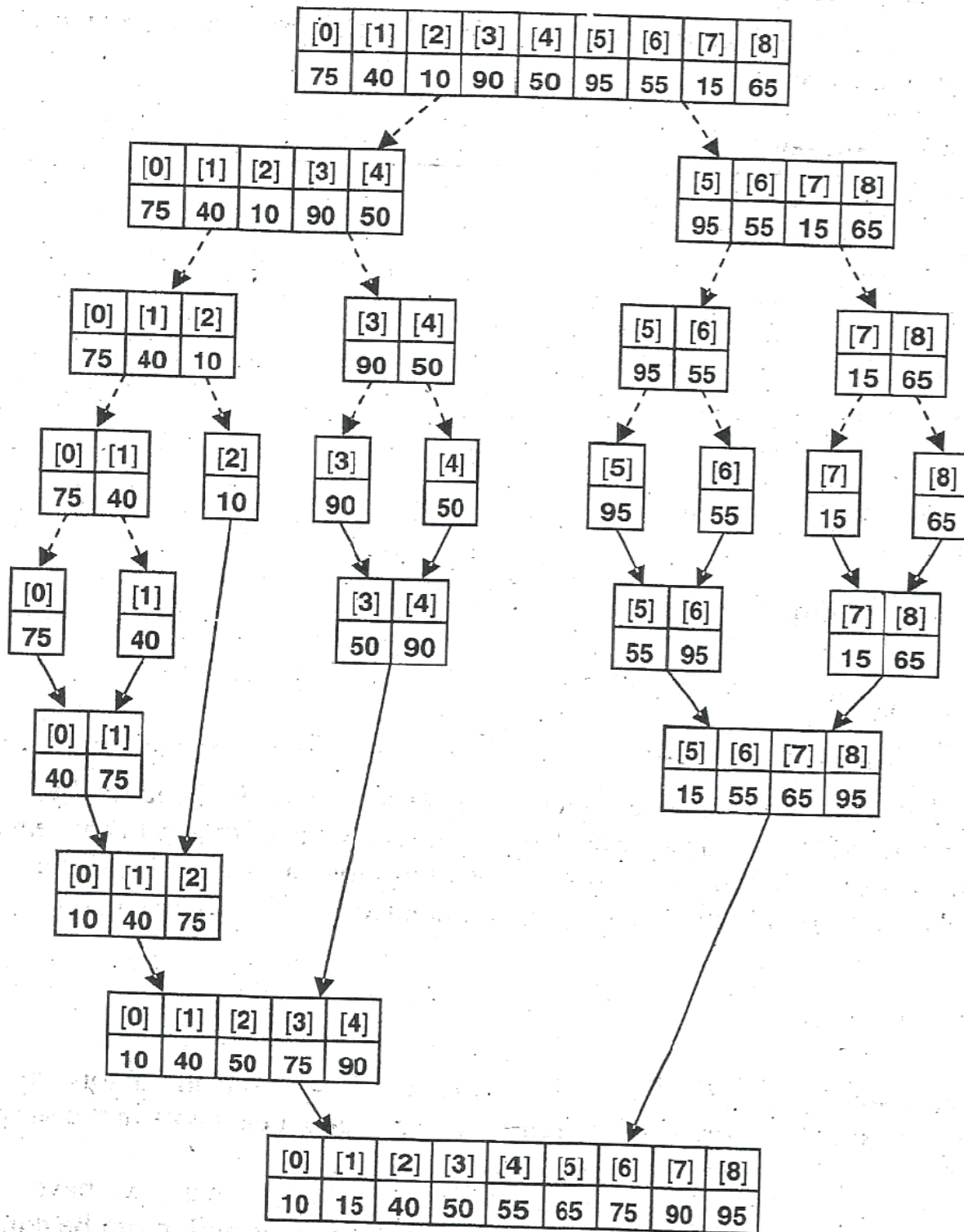


Figure 12.1: Two-way merge sort

Insertion Sort

- The insertion sort algorithm sorts the list by **moving** each element to its **proper place** in the **sorted portion** of the list.

```
void insertionSort(elemType list[], int length)
{
    for (int firstOutOfOrder = 1; firstOutOfOrder < length; firstOutOfOrder++)
        if (list[firstOutOfOrder] < list[firstOutOfOrder - 1])
        {
            elemType temp = list[firstOutOfOrder];
            int location = firstOutOfOrder;
            do
            {
                list[location] = list[location - 1];
                location--;
            }
            while (location > 0 && list[location - 1] > temp);
            list[location] = temp;
        }
} //end insertionSort
```

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
list	10	18	25	30	23	17	45	35

	← sorted list →				← unsorted list →			
	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
list	10	18	25	30	23	17	45	35

Insertion Sort

- Insertion sort consists of $N-1$ **passes**.
- For pass $p=1$ through $N-1$, insertion sort ensures that the elements in positions 0 through p are in sorted order.
- Insertion sort makes use of the fact that elements in positions 0 through $p-1$ are already known to be in sorted order.
- Figure 7.1 shows a sample array after each pass of insertion sort.
- Figure 7.1 shows the general strategy. In pass p , we move the element in position p left until its correct place is found among the first $p+1$ elements.
- Data movement implementation: The element in position p is moved to tmp, and all larger elements (prior to position p) are moved one spot to the right. Then tmp is moved to the correct spot.

Original	34	8	64	51	32	21	Positions Moved
After $p = 1$	8	34	64	51	32	21	1
After $p = 2$	8	34	64	51	32	21	0
After $p = 3$	8	34	51	64	32	21	1
After $p = 4$	8	32	34	51	64	21	3
After $p = 5$	8	21	32	34	51	64	4

Figure 7.1 Insertion sort after each pass

Insertion Sort Examples:

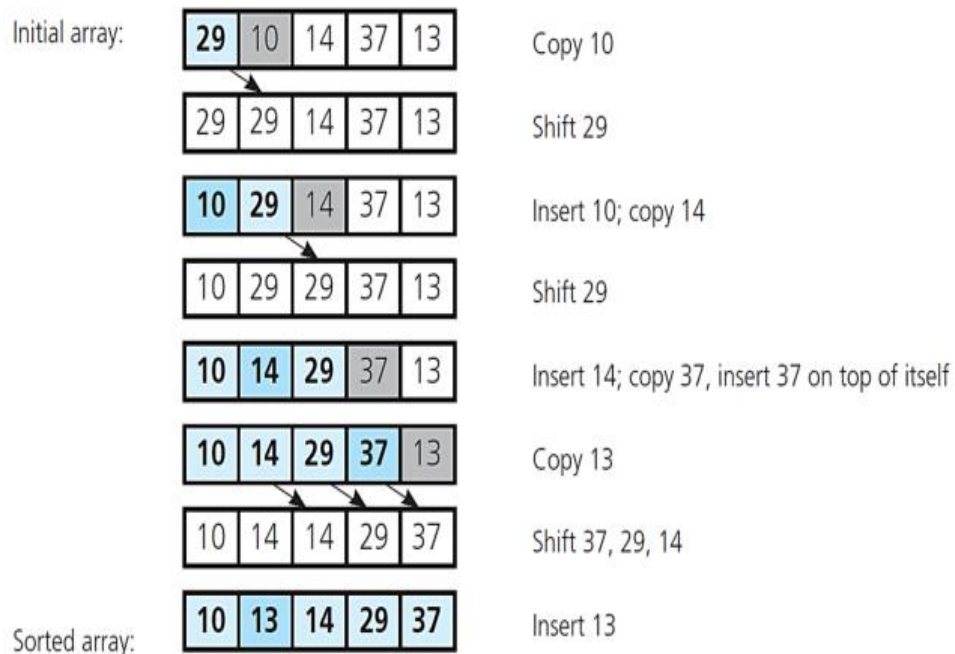
Example 1: Original Array: 29, 10, 14, 37, 13

Solution:

Original	29	10	14	37	13	Positions Moved
After p=1	10	29	14	37	13	1
After p=2	10	14	29	37	13	1
After p=3	10	14	29	37	13	0
After p=4	10	13	14	29	37	3

Illustration of how it is implemented:

An insertion sort of an array of five integers



Example 2: Original Array: (5, 6, 2, 4, 7, 3, 1).

Solution:

Original	5	6	2	4	7	3	1	Positions Moved
After p=1	5	6	2	4	7	3	1	0
After p=2	2	5	6	4	7	3	1	2
After p=3	2	4	5	6	7	3	1	2
After p=4	2	4	5	6	7	3	1	0
After p=5	2	3	4	5	6	7	1	4
After p=6	1	2	3	4	5	6	7	6

5	6	2	4	7	3	1
5	6	2	4	7	3	1
2	5	6	4	7	3	1
2	4	5	6	7	3	1
2	4	5	6	7	3	1
2	3	4	5	6	7	1
1	2	3	4	5	6	7

Insert 5

Insert 6

Insert 2

Insert 4

Insert 7

Insert 3

Insert 1

Quick Sort

- The quick sort algorithm uses the **divide-and-conquer** technique to sort a list.
- The list is **partitioned into two sublists**, and **combined** into one list in such a way so that the combined list is sorted.

```
void recQuickSort(elemType list[], int first, int last)
{
    int pivotLocation;
    if (first < last)
    {
        pivotLocation = partition(list, first, last);
        recQuickSort(list, first, pivotLocation - 1);
        recQuickSort(list, pivotLocation + 1, last);
    }
} //end recQuickSort

void quickSort(elemType list[], int length)
{
    recQuickSort(list, 0, length - 1);
} //end quickSort

int partition(elemType list[], int first, int last)
{
    elemType pivot;
    int index, smallIndex;
    swap(list, first, (first + last) / 2);
    pivot = list[first];
    smallIndex = first;
    for (index = first + 1; index <= last; index++)
        if (list[index] < pivot)
        {
            smallIndex++;
            swap(list, smallIndex, index);
        }
    swap(list, first, smallIndex);
    return smallIndex;
} //end partition
```